

AI Analyst — Modeling Reference

Every quantitative and machine-learning model in the app: what it does, the math, and how it is used.

Document date: 2026-07-28. Companion to [ai_analyst_documentation.pdf](#) and [qlib_integration.pdf](#).

Notation: w = portfolio weights, μ = expected returns (vector), Σ = covariance matrix, R_f = risk-free rate, $E[\cdot]$ = expectation, z = cross-sectional z-score. All annualization uses 252 trading days unless noted.

Part I — The qlib modeling approach

1. The qlib pipeline

qlib models an investment process as a chain, each stage configurable and independently usable:

```
DataLoader -> DataHandler -> Dataset -> Model -> Signal -> Strategy -> Backtest / Analysis
```

- **DataHandler / Dataset** turn a raw table into a supervised learning problem: a MultiIndex (`datetime`, `instrument`) frame of **features** X and a **label** y , split into train/valid/test date segments.
- **Model** learns $f: X \rightarrow \hat{y}$; its prediction $\hat{y}$ is the **signal** (a cross-sectional score per stock per date — here, an expected forward return).
- **Strategy + Backtest** turn signals into positions and simulate P&L.

How we use it (lean embed). We supply our own panel via `DataHandlerLP.from_df` (no qlib data provider, no `qlib.init`), then use qlib's `Model`, risk model, optimizers, and evaluation as plain libraries. Stages we *do not* use: the expression/.bin data engine, the recorder/`qrun` workflow, and the built-in backtest exchange (which needs the data provider).

2. Feature / label panel

`api/quant/qlib_data.build_panel` builds a **monthly, cross-sectional** panel:

- **Features X :** importance-ranked fundamentals in four families — *value*, *quality*, *growth*, *market-factor* — read from `fact_metrics_*` and **z-scored within each month** ($z = (x - \text{mean}_t) / \text{std}_t$, winsorized to ± 3). Cross-sectional z-scoring makes each factor comparable across names and dates and robust to scale.
- **Label y :** the **forward 1-month** (or 3-month) total return of each stock.
- **Point-in-time alignment:** a fundamental is stamped to `period_end + ~90 days -> month-end`, so a row only ever uses data that was public at that date (no look-ahead).

3. The alpha model — cross-sectional LightGBM (return prediction)

`api/quant/qlib_alpha.AlphaLGB` wraps qlib's `LGBModel` (LightGBM).

Method — gradient-boosted decision trees. LightGBM fits an additive ensemble of regression trees $F(x) = \sum_m \eta \cdot h_m(x)$, where each tree h_m is fit to the negative gradient (for the L2/mse objective, the residual $y - F_{\{m-1\}}(x)$) of the loss, η is the learning rate, and leaves are grown with the histogram/leaf-wise algorithm. Trees capture **non-linearities and interactions** among factors that a linear factor model cannot.

Training. `fit()` uses the qlib `DatasetH` train segment, early-stops on the valid segment's L2, with regularization (`num_leaves`, `max_depth`, `lambda_l1/l2`, feature/row subsampling). The subclass removes qlib's mlflow recorder logging so training needs no `qlib.init`.

Prediction. The booster scores the latest cross-section $\rightarrow$ an expected forward return per stock. Annualize by $\times 12$ (monthly horizon) to feed an optimizer.

Evaluation — Information Coefficient. Per date t , $IC_t = \text{corr}(\hat{y}_t, y_t)$ (Pearson) and $\text{RankIC}_t = \text{Spearman}(\hat{y}_t, y_t)$. We report the mean and $\text{ICIR} = \text{mean}(IC) / \text{std}(IC)$. RankIC ≈ 0.05 – 0.10 is a usable monthly equity signal; the shipped US model reaches ≈ 0.09 – 0.14 out-of-sample.

4. The risk model — StructuredCovEstimator (forward covariance)

`api/quant/qlib_risk` wraps qlib's `StructuredCovEstimator`. It assumes returns are driven by a small number of **latent factors**:

```
X = B · F^T + U      (observations x variables = returns panel)
cov(X) = F · cov(B) · F^T + diag(var(U))
```

- **F** (variables × factors) = factor **exposures**, from **PCA** (principal components) or **Factor Analysis** on the return panel.
- **B** (observations × factors) = factor **returns** (the PCA scores); `cov_b = cov(B)`.
- **U** = idiosyncratic residual; `var_u = var(U)` is the diagonal specific variance.

This shrinks a noisy $N \times N$ sample covariance to a low-rank + diagonal form (far fewer parameters, more stable for optimization). We construct it with `scale_return=False` (qlib's default rescales returns to percent, inflating variance 100²×) and annualize `cov_b`, `var_u` by ×252. Forward volatility per name is $\sqrt{\text{diag}(\Sigma)}$. The (`F`, `cov_b`, `var_u`) decomposition feeds the enhanced-indexing optimizer directly.

5. The qlib optimizers

All are long-only and fully invested ($\Sigma w = 1$).

Method	Objective
GMV (global min variance)	$\min_w w^T \Sigma w$
MVO (mean-variance)	$\max_w w^T \lambda \cdot r - w^T \Sigma w$ (r = return scaled to Σ 's vol; λ = risk-aversion)
RP (risk parity)	choose w so each name's risk contribution $w \cdot (\Sigma w)$ is equal
INV (inverse volatility)	$w \propto 1 / \sigma$

GMV/MVO/RP are solved with `scipy.optimize`; turnover (`delta`) and an L2 term (`alpha`) can be added.

Enhanced Indexing (benchmark-relative, cvxpy/ECOS). With $d = w - w_b$ (deviation from benchmark) and $v = dTF$ (factor deviation):

```
max_w dTr - λ · ( vT · cov_b · v + var_u · d^2 )
s.t.    w ≥ 0,    Σw = 1,    ||w - w0|| ≤ delta,    |d| ≤ b_dev,    |v| ≤ f_dev
```

i.e. maximize active return minus active (tracking-error) risk, holding factor and name deviations within bounds — the standard "beat the index with controlled tracking error" formulation.

6. Signal backtest

`api/quant/qlib_backtest` runs a cross-sectional **signal backtest** (qlib's exchange backtest needs the data provider, so it is not used): each month, rank by predicted alpha, form an equal-weight top-k (optionally long/short) book, realize the panel's forward-return label, and score the monthly return series with qlib's pure **risk analysis** (annualized return, information ratio, max drawdown) plus `calc_ic`.

Part II — Non-qlib models

A. Valuation models (`ai_analyst/`)

A.1 Discounted Cash Flow (`dcf_engine.py`)

A transparent 7-year FCFF DCF. For each forecast year *i*:

```
Revenue_i = Revenue_{i-1} · (1 + g_i)
EBIT_i     = Revenue_i · ebit_margin
NOPAT_i    = EBIT_i - max(EBIT_i, 0) · tax_rate
FCF_i      = NOPAT_i + D&A_i - Capex_i - ΔNWC_i      (D&A, Capex, ΔNWC as % of revenue)
```

Terminal value (Gordon growth) and enterprise/equity value:

```
TV = FCF_7 · (1 + g_term) / (WACC - g_term)
EV  = Σ_i FCF_i / (1+WACC)^i + TV / (1+WACC)^7
Equity = EV - net_debt ; per_share = Equity / shares
```

Assumptions come from the committee's scenario LLM (bounded/normalized); a **5×5 sensitivity grid** varies WACC (±200 bp) × terminal growth (±100 bp).

A.2 Cost of capital — WACC ([committee/wacc.py](#))

Cost of equity is built bottom-up from **Fama-French factor betas** and long-run factor premia:

$$\begin{aligned} \text{Re}(\text{CAPM}) &= R_f + \beta_{\text{mkt}} \cdot \text{ERP} \\ \text{Re}(\text{FF5}) &= R_f + \beta_{\text{mkt}} \cdot \text{ERP} + \beta_{\text{smb}} \cdot \text{SMB} + \beta_{\text{hml}} \cdot \text{HML} + \beta_{\text{rmw}} \cdot \text{RMW} + \beta_{\text{cma}} \cdot \text{CMA} \quad (+ \beta_{\text{mom}} \cdot \text{MOM}) \end{aligned}$$

R_f = 10Y Treasury (FRED DGS10); premia = long-run FF factor averages ×252; betas from the rolling factor regression (\$B.1). Cost of debt $R_d = R_f + \text{credit_spread}$, where the spread comes from a Damodaran-style synthetic-rating table keyed on interest coverage $\text{EBIT}/\text{Interest}$ (blended with the realized $\text{interest}/\text{debt}$ rate when plausible). Then:

$$\text{WACC} = (E/V) \cdot \text{Re} + (D/V) \cdot R_d \cdot (1 - \text{tax_rate})$$

E = market cap, D = total financial debt. The headline uses $\text{Re}(\text{CAPM})$ (stable); the full FF5 Re is reported as an exhibit. `segment_wacc()` shifts beta / adds a growth-risk premium for the SOTP.

A.3 Reverse DCF ([committee/valuation.py](#))

Inverts the DCF by **bisection** to answer "what is the market pricing in?":

- **Reverse growth:** hold everything at base, solve for the flat revenue growth g^* such that $\text{per_share}(g^*) = \text{current_price}$.
- **Reverse margin:** freeze the growth path, solve for the steady EBIT margin m^* that justifies the price — the sharpest read on the operating leverage / pricing power the market is underwriting (flagged if it exceeds best-in-class peer margins).

A.4 SOTP, scenarios, triangulation

- **Sum-of-the-parts:** value each business segment with its own DCF and a **segment-specific WACC** (higher for higher-growth/beta segments), then aggregate — the declared *primary* fair value.
- **Scenario weighting:** $\text{fair_value} = \sum_s \text{weight}_s \cdot \text{per_share}_s$ over upside/base/downside; weights are fixed or macro-adjusted and must sum to 1.
- **Triangulation** (`triangulate`): assembles a **football field** from SOTP (primary), the consolidated DCF scenario range, and peer-multiples-implied values, and reports `implied_upside_pct` vs price.

B. Factor & portfolio-risk models ([api/quant/](#), [xbrl_sec/](#))

B.1 Fama-French rolling factor regression ([xbrl_sec/sec/sources/factor_model.py](#))

Per ticker, over a rolling **252-day** window, regress excess returns on the FF factors (FF3/FF4/FF5/FF6):

$$r_i - R_f = \alpha + \sum_k \beta_k \cdot \text{factor}_k + \epsilon$$

- **Robust estimation:** Huber regression (down-weights outlier days) rather than plain OLS.
- **Standard errors:** **Newey-West** HAC covariance (corrects for autocorrelation/heteroskedasticity).
- **Diagnostics stored:** $\text{adj_R}^2 = 1 - (1 - R^2)(n-1)/(n-k)$, residual vol = $\text{std}(\epsilon) \cdot \sqrt{252}$ (the idiosyncratic risk D used elsewhere), Durbin-Watson, condition number, F-stat. Loadings feed `fact_factor_loadings` (WACC betas, structured covariance) and per-day model-implied returns.

B.2 Ledoit-Wolf shrinkage covariance ([api/quant/risk.py](#))

Shrinks the sample covariance S toward a **constant-correlation** target F (mean off-diagonal correlation $r_{\blacksquare}, F_{ij} = r_{\blacksquare} \cdot \sigma_{\blacksquare} \sigma_{\blacksquare}, F_{ii} = \sigma_{\blacksquare}^2$):

$$\Sigma_{\text{shrunk}} = \delta \cdot F + (1 - \delta) \cdot S, \quad \delta = \text{clamp}(\pi / \gamma, 0, 1)$$

π = sum of asymptotic variances of the entries of S ; $\gamma = \blacksquare S - F_{\blacksquare}^2$. Optimal δ trades sample noise against target bias; result annualized ×252.

B.3 Fama-French structured covariance ([api/quant/risk.py](#))

$\Sigma = B \cdot F \cdot B^T + D$, where B = FF loadings (`fact_factor_loadings`), F = factor-return covariance ×252, and $D = \text{diag}(\text{residual_vol}^2)$ from the regression. A **fundamental** (named-factor) counterpart to qlib's statistical (PCA) structured covariance.

B.4 Native portfolio optimizer ([api/quant/optimize.py](#))

Maximizes a multi-term utility, solved with **SLSQP** (analytic gradient):

```
max_w wTμ - ½λ·wTΣw - γ_f·S·a·(BTw - t)² - γ_to·w - wθ - γ_c·wTw
```

subject to: $\sum w = 1$; per-name bounds; gross cap $\sum |w| \leq \text{gross_max}$; short-leg cap; sector caps; an **annualized vol cap** $wT\Sigma w \leq \sigma_{\text{max}}^2$; and factor caps/ranges $|BTw|_j \leq \text{cap}$. When a **cardinality** constraint (`max_names`) is set, it switches to a **mixed-integer program** (binaries $z_i, w_i \leq z_i, \sum z_i \leq \text{max_names}$) solved via cvxpy (HiGHS/SCIP/CBC). It also sweeps the **efficient frontier** and reports marginal risk contributions. This is the **native** backend in the runtime optimizer selector.

B.5 Portfolio risk metrics ([api/routers/portfolio.py](#))

From the portfolio NAV / return series:

```
Sharpe      = (μ_annual - Rf) / σ_annual
Max drawdown = min_t ( NAV_t / max_{s≤t} NAV_s - 1 )
Beta        = cov(R_p, R_b) / var(R_b) ; Corr = corr(R_p, R_b) (benchmark default SPY)
Effective N  = 1 / Σ w² (concentration)
Historical VaR_α / CVaR_α : the α-quantile of the empirical return distribution (VaR) and the mean of the
                           tail beyond it (CVaR), over several windows × α ∈ {95%, 99%, 99.9%}.
```

C. Macro-cycle & signal models ([xbr1_sec/sec/cycle/](#))

C.1 Compressed cycle factors (baseline)

A PCA / dynamic-factor baseline compresses a multimodal feature set (macro, market, fundamental series) into a handful of **latent cycle factors** stored per month ([fact_cycle_state_monthly](#), [latent_cycle](#)). These are the inputs to the regime models below.

C.2 HMM regime model ([cycle/hmm.py](#))

A **Gaussian Hidden Markov Model** ([hmmlearn](#), `n_states = 4`, full covariance) over the latent cycle factors learns:

- state **means** and **covariances** (each hidden state = a market regime),
- a **transition matrix** $P(\text{state}_t \mid \text{state}_{t-1})$,
- per-month posterior **state probabilities** $P(\text{state} \mid \text{data})$ and the most-likely state.

States are mapped to interpretable business-cycle **phases** (e.g. early/mid expansion, slowdown, contraction, recovery) via a calibration on the factor series. Output per month: phase label + phase probabilities + confidence ([max](#) posterior). A deterministic numpy k-means-style fallback is used if [hmmlearn](#) is unavailable.

C.3 VAE regime model ([cycle/vae.py](#))

An optional **temporal multimodal variational autoencoder** (PyTorch): an encoder maps the multimodal features to a low-dimensional latent **z** (dimensions split into interpretable **growth / inflation / rates-liquidity / credit-stress / market / fundamentals** latents), and a decoder reconstructs the inputs. The **reconstruction-error stress percentile** is smoothed and thresholded into cycle phases with a confidence. Per-modality reconstruction error attributes stress to macro vs market vs fundamentals. If PyTorch is absent, a PCA surrogate produces the same output shape.

C.4 Regime-conditioned factor IC ([cycle/ic.py](#))

The link from the regime model to forecasting. For each fundamental factor (`metric_id`), **conditioned on the regime**, compute the rank information coefficient against forward returns:

```
IC_regime(metric) = Spearman( metric_value , forward_return | regime )
```

A **probability-weighted** variant weights each observation by the regime's posterior probability (so months are not hard-assigned to one regime). Results ([fact_equity_factor_ic_regime](#)) say *which factors have actually predicted returns in conditions like today's*, and drive the scanner's factor-family weights (§D.1, and the "macro-regime IC in forecasting" section of [qlib_integration.pdf](#)).

D. Scoring & ranking models

D.1 Value + Sentiment interest score (api/routers/screener_agent.py)

A 0–100 cross-sectional score. Each component is min-max normalized across the shortlist to [0,1] (v_fcf, v_pe (inverted — cheaper better), v_g, v_tone, v_news), plus v_alpha = the normalized qlib expected return:

```
score = w_alpha · v_alpha + (1 – w_alpha) · Σ_k w_k · v_k ; interest = 100 · score
```

w_alpha (default 0.40) is the dominant term when the alpha model covers the shortlist. The fundamental sub-weights w_k come from fixed availability-branched defaults, re-split between value and growth by the current regime's factor IC (§C.4). With no model / IC available it reduces exactly to the legacy fixed-weight formula.

D.2 Group relative-value composite (committee/group.py)

A deterministic ranking. For each metric k (P/E, EV/EBITDA, P/B, FCF yield, dividend, revenue growth/CAGR, gross/operating margin), compute a cross-sectional z-score z_k within the peer group and a signed weight w_k (negative for "cheaper-is-better" multiples). Loss-making multiples (negative P/E, EV/EBITDA, P/B) are excluded from their metric's z-score population rather than ranked as "cheap".

```
composite(name) = Σ_k w_k · z_k(name)
```

Names are ordered by the composite; terciles map to attractive / fair / expensive. Every name carries a full audit trail (value → z → xweight → contribution) that sums exactly to the composite.

D.3 MD&A management-tone (LLM)

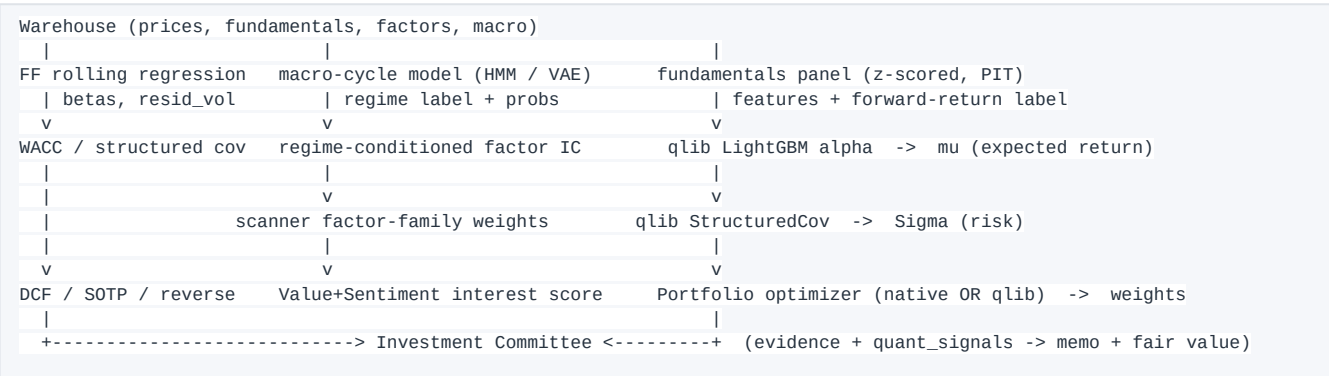
An LLM reads an excerpt of each name's latest MD&A and returns a structured tone in [-1, 1], a positive/neutral/negative guidance label, and risk flags. The tone feeds v_tone in §D.1.

E. The committee as a model (ai_analyst/committee/)

The multi-agent tribunal is a reasoning system, not a numeric model, but it is disciplined:

- **Structured extraction** — scenario assumptions, specialist verdicts, and the ranked group verdict are produced via JSON-mode structured outputs validated against pydantic schemas; the deterministic engine (§A) turns those assumptions into numbers.
- **Deliberation** — Advocate / Challenger / Auditor + specialists argue over the same evidence packet; the Lead synthesizes and may loop (≤ 3 rounds). Multiple LLM providers can debate the same prepared evidence concurrently.
- **Probability-weighted valuation** — the numeric output (scenario-weighted fair value, triangulation) is deterministic given the LLM's assumptions, so the "why" of every figure is auditable.

Part III — How the models connect



The **valuation** path (DCF/WACC/SOTP/reverse/triangulation) answers *what a business is worth*; the **quant** path (alpha/risk/optimization) answers *what to expect and how to size it*; the **macro-cycle** path conditions both on *the regime we are in*; and the **committee** fuses the evidence into a reasoned, auditable verdict.